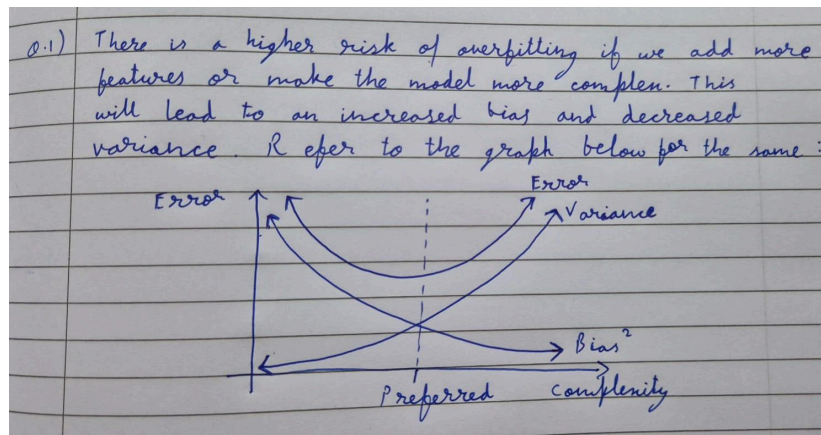


ML Assignment 1

Parth Garg (2022351)

Question 1 (Theoretical)

Part A:



Part B:

Q.2)

$$\begin{aligned} TP &= 200 \\ FP &= 20 \\ TN &= 730 \\ FN &= 50 \end{aligned}$$
$$\text{Spam Precision} = \frac{TP}{TP + FP} = \frac{200}{220} = 0.91$$
$$\text{Spam Recall} = \frac{TP}{TP + FN} = \frac{200}{250} = 0.8$$
$$\text{Legitimate Precision} = \frac{TN}{TN + FP} = \frac{730}{750} = 0.973$$
$$\text{Legitimate Recall} = \frac{TN}{TN + FN} = \frac{730}{780} = 0.936$$
$$\text{Spam F1 score} = \frac{2(\text{Precision})(\text{Recall})}{\text{Precision} + \text{Recall}} = 0.85$$
$$\text{Legitimate F1 score} = \frac{2(0.936)(0.973)}{0.936 + 0.973} = 0.95$$
$$\text{Average F1 score} = 0.9$$

Part C:

Q-3)	x	y	x^2	xy
	3	15	9	45
	6	30	36	180
	10	55	100	550
	15	85	225	1275
	<u>18</u>	<u>150</u>	<u>324</u>	<u>1800</u>
	52	285	694	3850

$$y = ax + b \quad \text{--- (1)}$$

$$a = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2} \quad \text{--- (2)}$$

$$= 5.783$$

$$b = \frac{\sum y \sum x^2 - \sum x \sum xy}{n \sum x^2 - (\sum x)^2} \quad \text{--- (3)}$$

$$= 3.146$$

③, ② in ①

$$y = 5.783x - 3.146$$

$$y = 66.25 \text{ for } x = 12$$

Part D:

a.4) Training set : $(0,0), (1,1), (2,4), (3,9)$

Let $f_1(x) = x^2$
 $f_2(x) = 4$

MSE of $f_1(x) = 0$

MSE of $f_2(x) = 12.5$

Testing Point : $(4,4)$

$f_1(4) = 16$
 $f_2(4) = 4$

MSE of $f_1(x) = 144$
MSE of $f_2(x) = 0$

f_1 fits training data perfectly, ~~and generalizes~~ but overestimates value for test. f_2 has low MSE for test but high for training. So, even though f_1 fits training data perfectly, it does not necessarily generalize better than f_2 .

Question 2 (Scratch Implementation)

To prepare the dataset for logistic regression, I began by handling missing values through data imputation. For categorical features, I filled the null values using the mode, and for numerical features, I imputed the missing values with the mean.

	0
male	0
age	0
education	105
currentSmoker	0
cigsPerDay	29
BPMeds	53
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	50
sysBP	0
diaBP	0
BMI	19
heartRate	1
glucose	388
HeartDisease	0

After imputation, I applied Z-score normalization to the numerical data. Without normalization, the initial graphs showed significant noise, making it difficult to identify clear convergence patterns. Although reducing the learning rate helped to some extent in mitigating the noise, the resulting confusion matrix was unsatisfactory, leading me to adopt Z-score normalization.

Then the data was split into 70:15:15 (train: test: validation) sets.

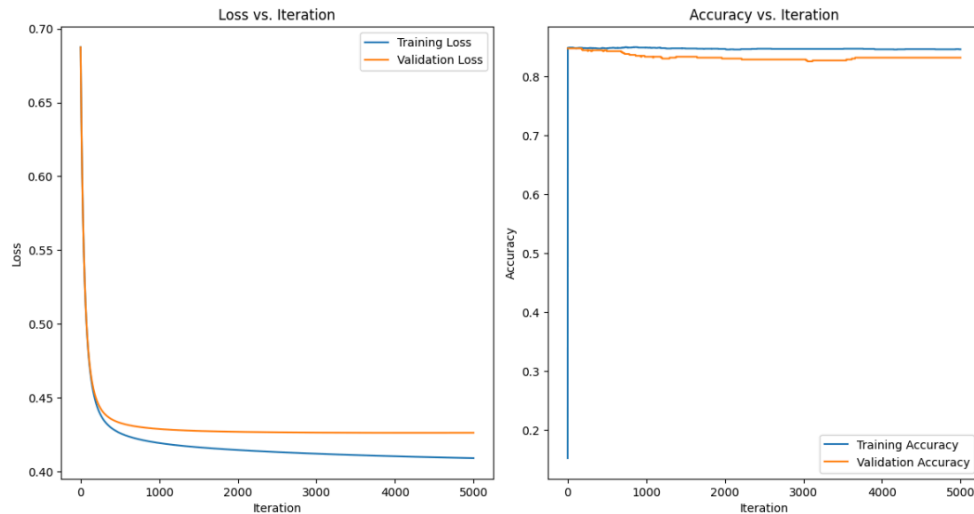
Part A:

I start by setting all weights to zero. I then create lists to keep track of how the training and validation losses and accuracies change at each step. Cross Entropy Loss is being used here to compute cost.

For each iteration:

- The outputs are predicted for the training data using the current weights and apply the sigmoid function to get probabilities.
- I calculate how far off the predictions are from the actual values, and based on this, I adjust the weights using the learning rate.
- After updating the weights, I calculate and store the training loss and accuracy.
- I do the same for the validation data to track the validation loss and accuracy.

I trained the model for 5000 iterations with a learning rate of 0.01. Every 1000 iterations, the training and validation losses are printed to monitor progress.

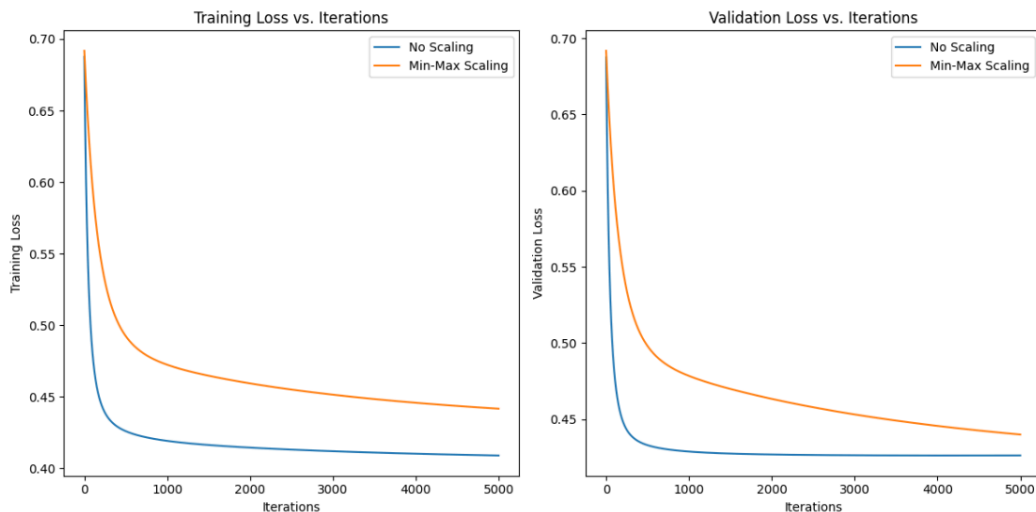


Final Validation Loss: 0.426229978732527
Final Validation Accuracy: 0.8317610062893082

The plots show the cross-entropy loss for both the training and validation sets at each iteration. For a learning rate of 0.01, the loss converges around 750 iterations. The validation loss converged before the training loss, suggesting that the model generalized well on unseen data early on. Both the training and validation losses steadily decreased before flattening out, indicating successful convergence and reliable model performance.

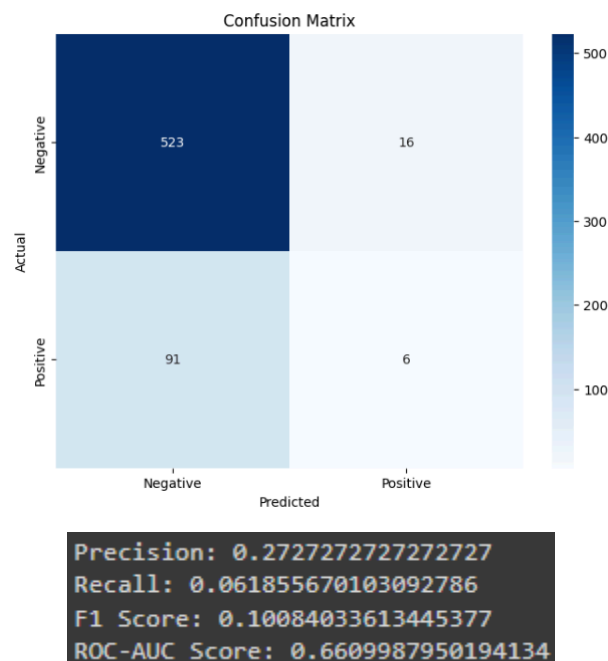
Part B:

After applying min-max scaling, the graphs were plotted and can be seen below. The model without scaling converged much earlier compared to min-max scaling, even with the same learning rate of 0.01 and 5000 iterations. This indicates that while min-max scaling normalizes the data, it slowed down the convergence process compared to the model with no scaling.



Part C:

No scaling was used for the confusion matrix below because it had better performance than min-max scaling.



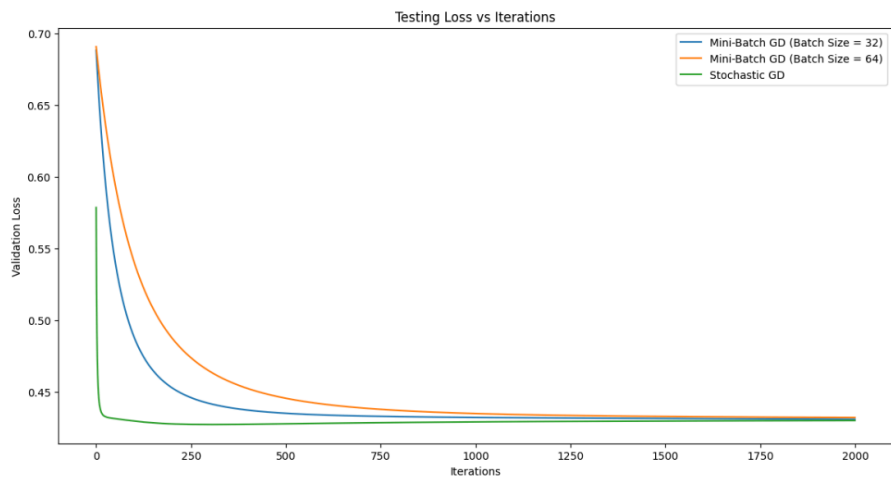
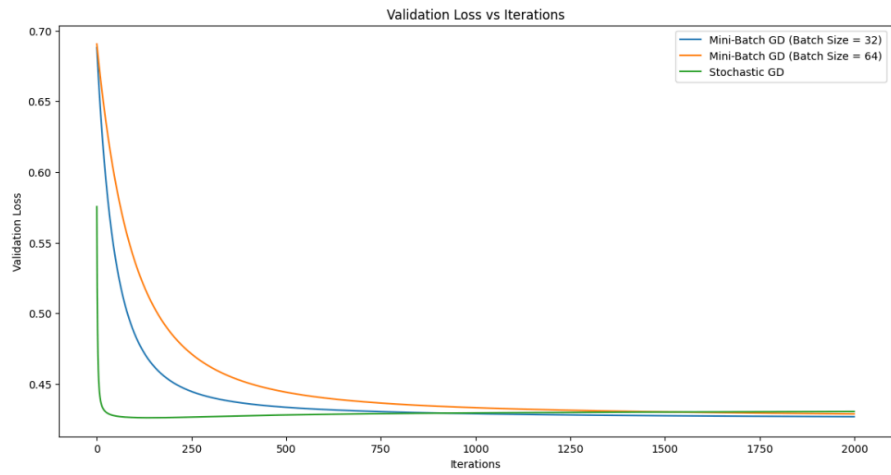
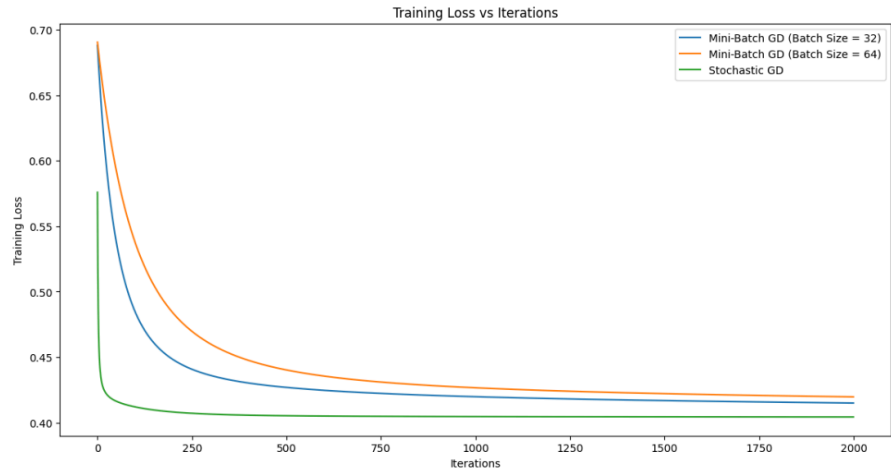
Looking at the metrics above:

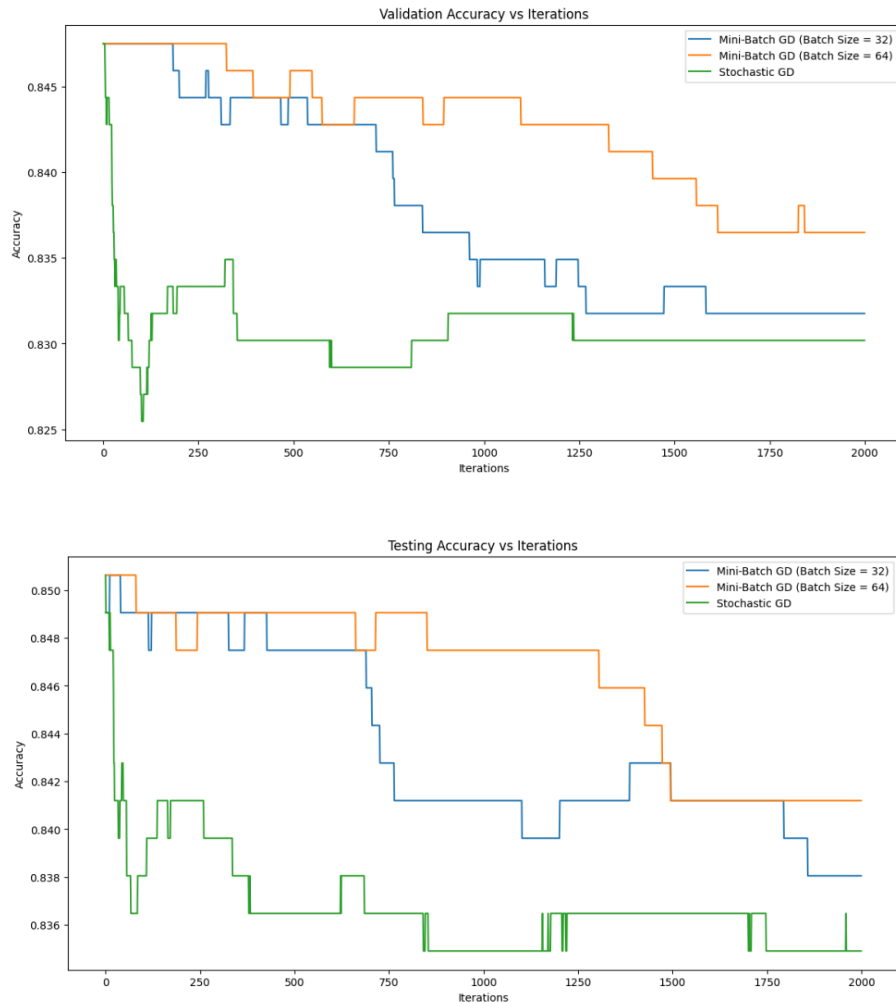
- The model correctly identifies positive cases only 27.3% of the time when it predicts them.
- The model only detects 6.2% of all actual positive cases.
- The model's overall performance in balancing precision and recall is very low according to the F1 score, indicating poor detection of positive cases.
- The model has moderate ability to distinguish between positive and negative classes according to the ROC-AUC score, but there is room for improvement.

These values can be justified by the fact that the model is biased towards the majority class (0), as it struggles to detect the minority class (1). There are only 644 1s in the dataset compared to 3594 0s. This is evident from the confusion matrix, where there are many false negatives and few true positives.

Part D:

The graphs for Loss and accuracy vs iterations for Stochastic Gradient Descent and Mini-Batch Gradient Descent (for batch sizes of 32 and 64) with a learning rate of 0.0001 and 2000 iterations can be found below:



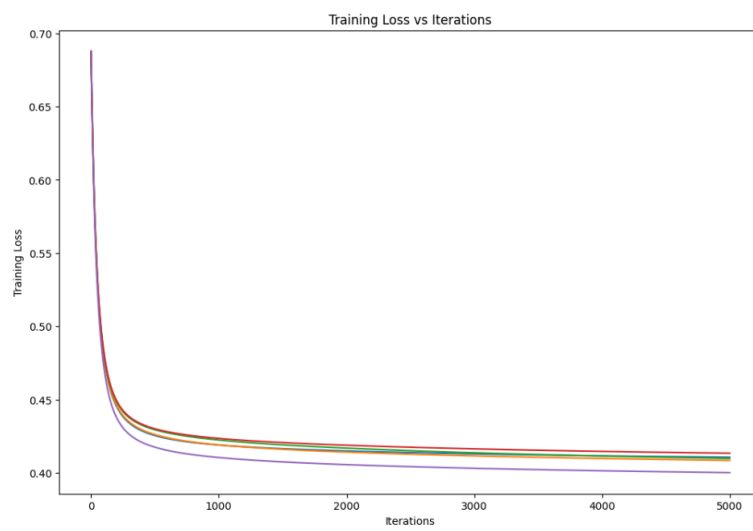
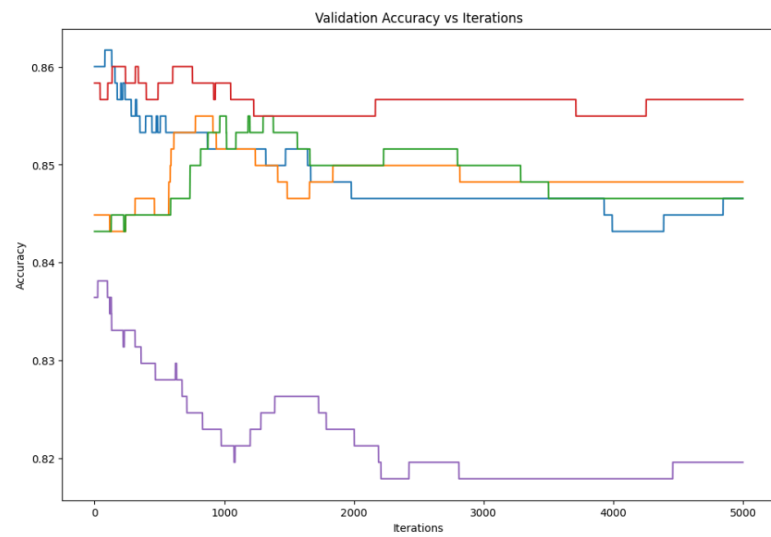
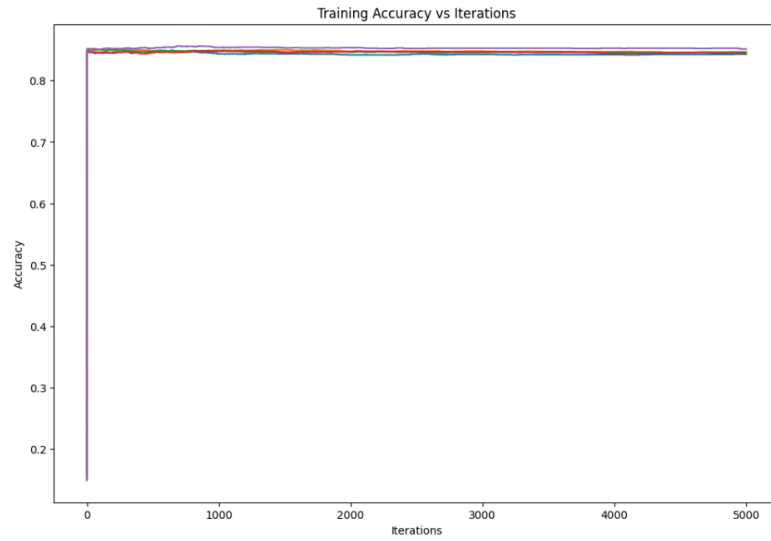


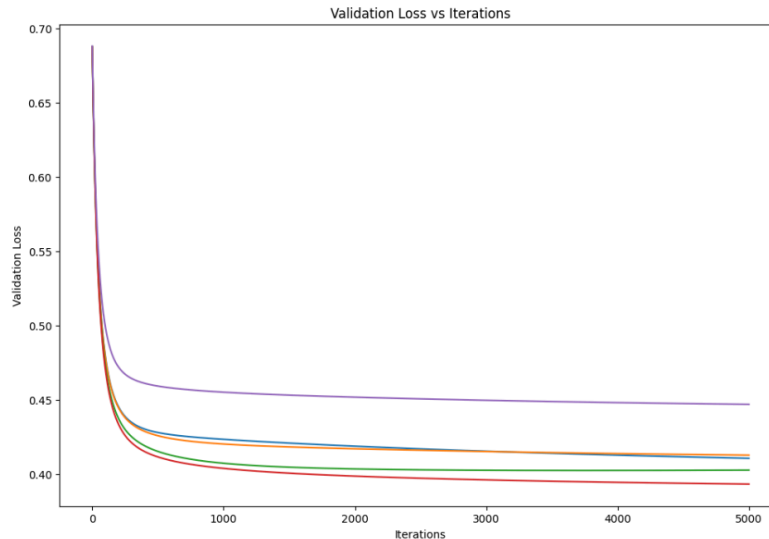
SGD converged the fastest but with high variance but had the lowest accuracy of the three. MBGD with batch size 32 converged faster than with batch size 64. It had accuracy between that of SGD and MBGD with batch size 64. MBGD with batch size 64 converged slower compared to batch size 32. It had the highest accuracy among the three methods.

Explaining the trade-off between Convergence Speed and Stability, models that reach convergence quickly often show increased loss and lower accuracy during later iterations. Despite this, all three models exhibited similar loss and accuracy on their validation and test sets.

Part E:

The following graphs display the loss and accuracy versus iterations for K-fold validation (K = 5 as asked) with a learning rate of 0.01 and 5000 iterations:





The model's performance shows:

The accuracy of the model is high and consistent with minimal fluctuation across folds, showing reliable performance.

Precision shows a moderate average with significant variability, indicating that the model's effectiveness in identifying positive cases differs between folds.

Recall is consistently low and stable, meaning the model infrequently detects positive cases but does so consistently.

The F1 Score, being low with moderate variance, reflects the combined effects of low precision and recall and how they are consistent across different folds.

In summary, while the model maintains stable accuracy, it exhibits variability in precision and F1 score, with consistently low recall.

```
Accuracy:
  Mean = 0.8435075885328838, Standard Deviation = 0.012547099654460908

Precision:
  Mean = 0.45235978835978835, Standard Deviation = 0.09912696139274474

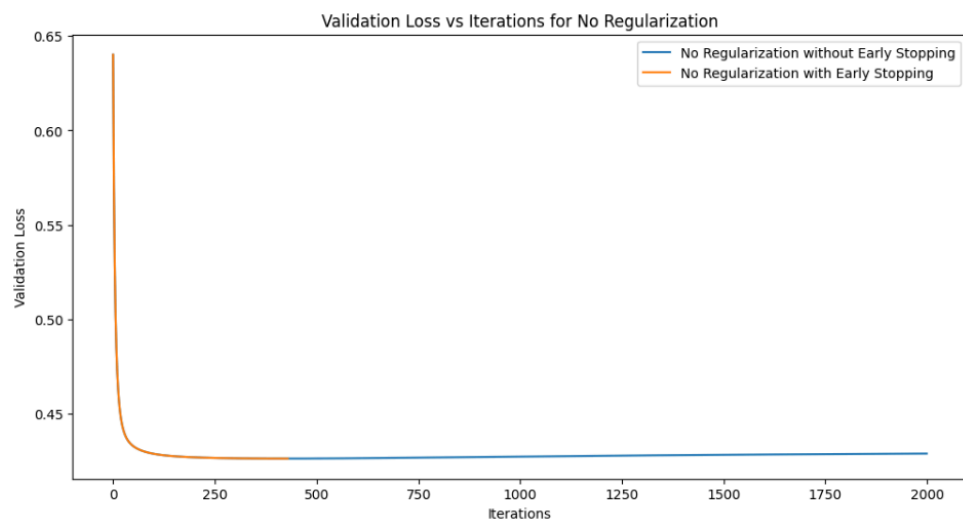
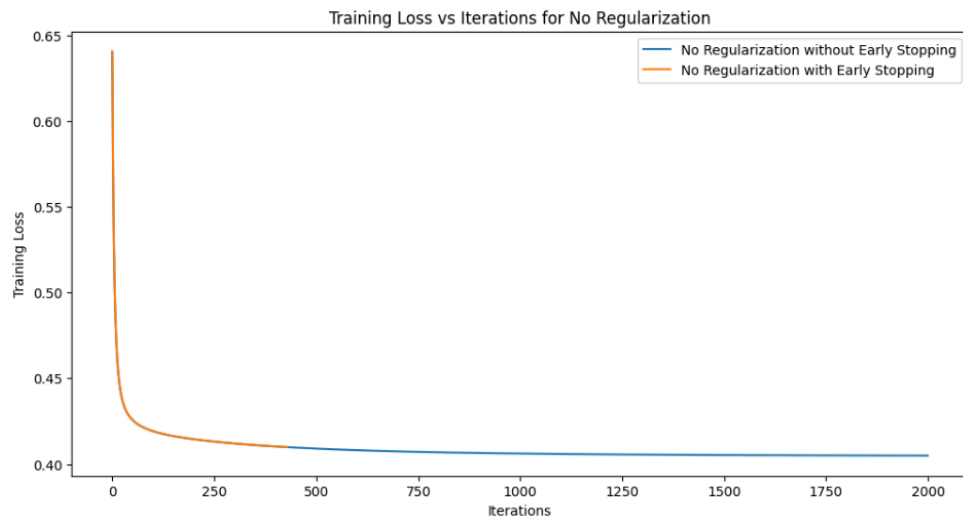
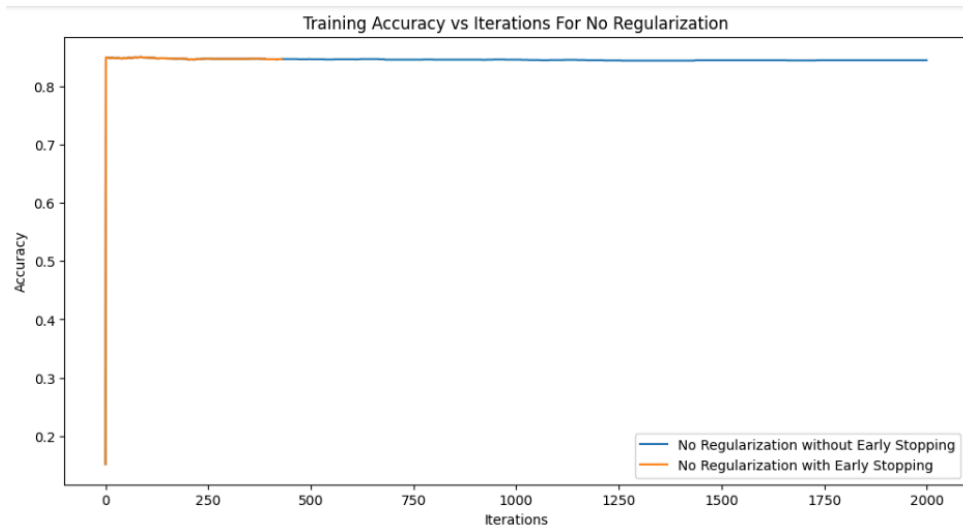
Recall:
  Mean = 0.10668420818390327, Standard Deviation = 0.016312625549698742

F1 Score:
  Mean = 0.17183823131388834, Standard Deviation = 0.02627340399080812
```

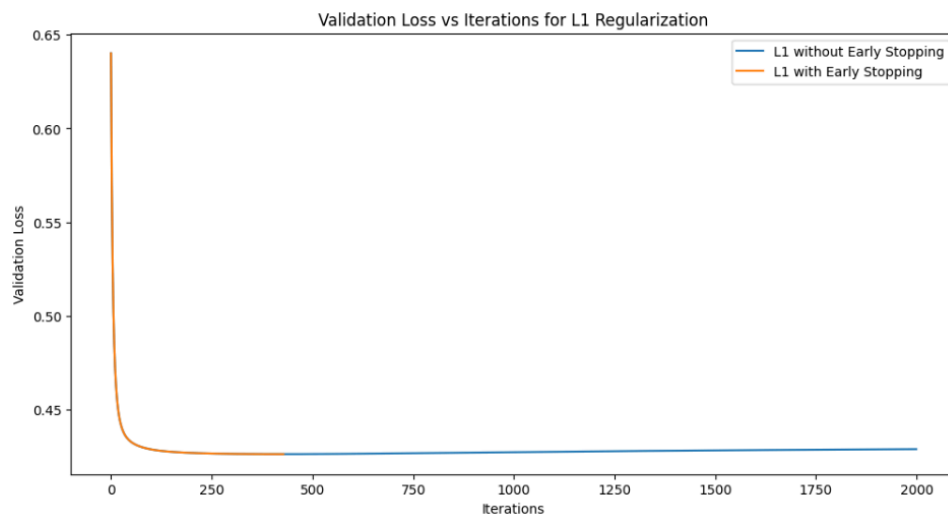
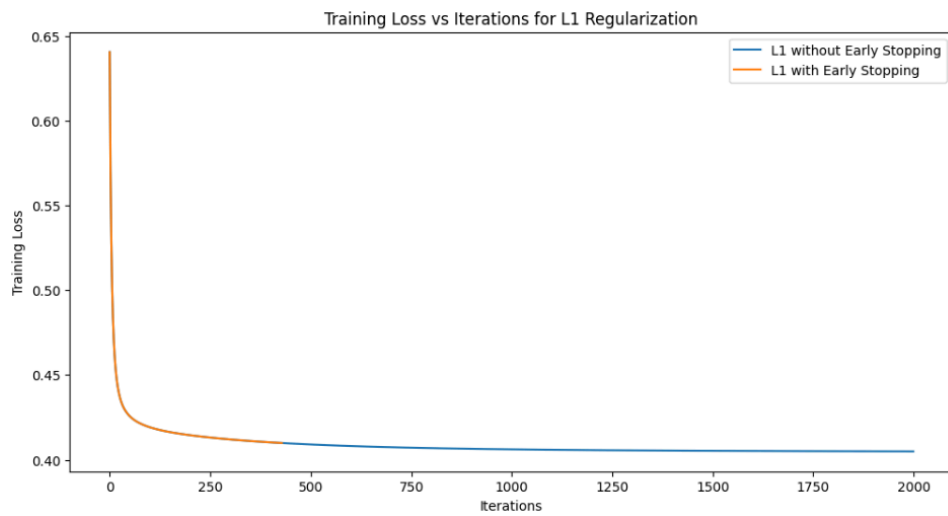
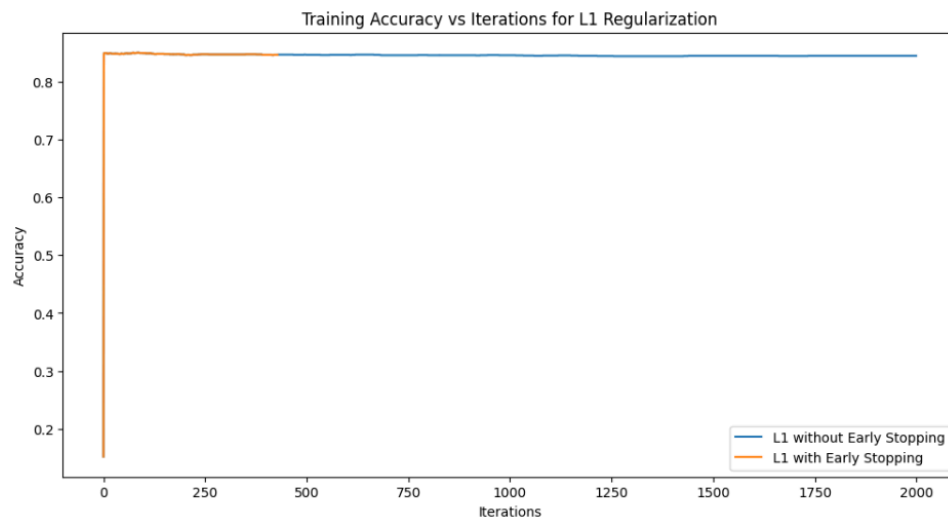
Part F:

Below are the plots for accuracy and loss versus iterations. These show the effects of early stopping (orange plot) compared to running without early stopping (blue plot) for each case with No, L1, and L2 regularization.

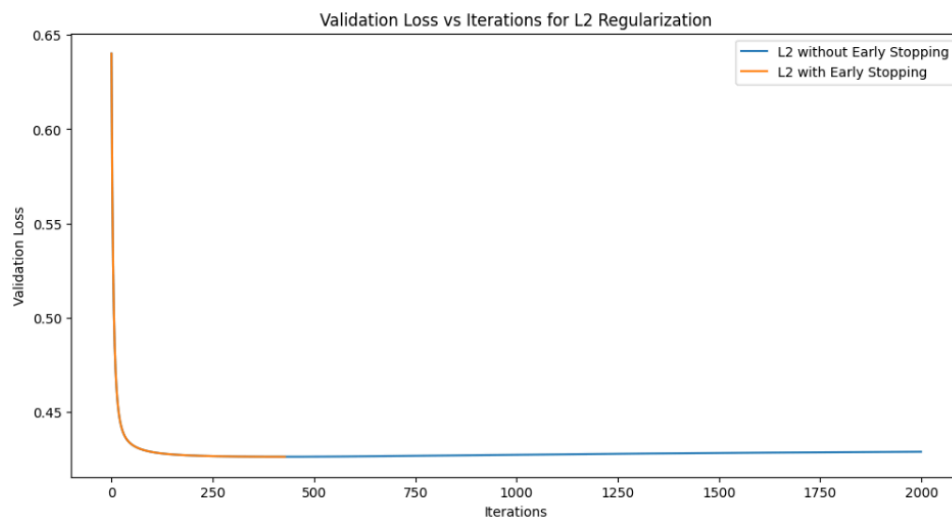
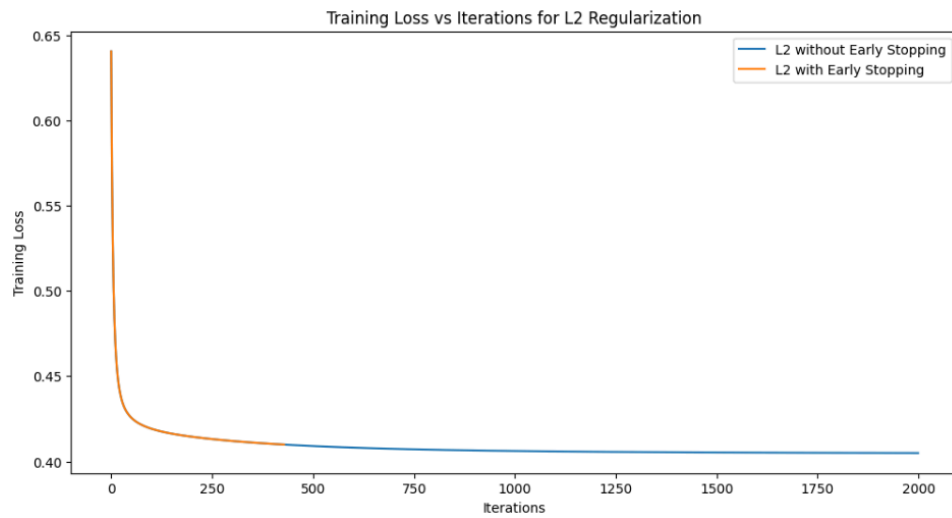
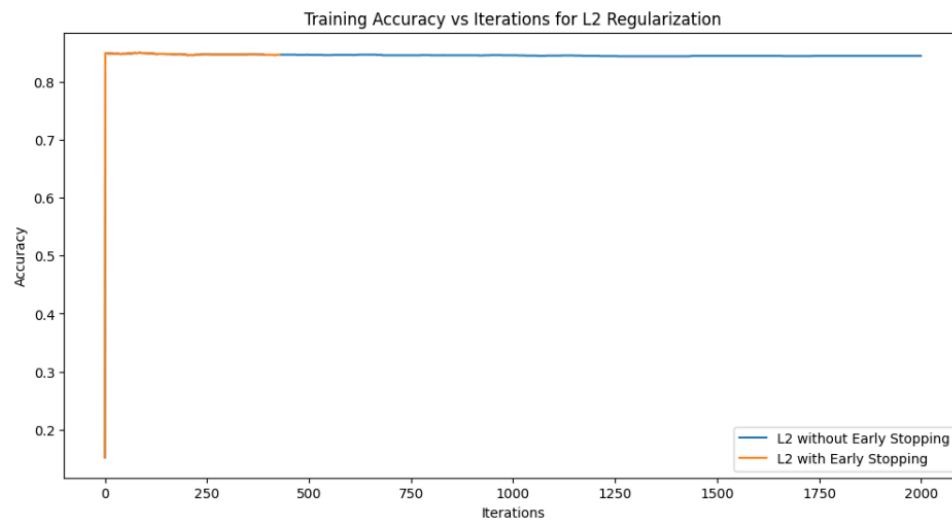
No Regularization:



L1 Regularization:



L2 Regularization:



The plots demonstrate that early stopping is effective, as training halts when the loss begins to increase, indicating that the model is no longer improving on the validation set. Overfitting is evident from the graphs, as the training loss continues to decrease while the validation loss starts to rise after the point where early stopping is applied. Despite this, the accuracy remains stable for both the training and validation sets, suggesting that generalization is not adversely affected by early stopping.

I am using a learning rate of 0.1, 2000 iterations, a regularization parameter (λ) of 0.1, and a patience of 10 for the output below:

```
Iteration 0, Training Cost: 0.6404310864296741, Validation Cost: 0.6400881793733778, Training Accuracy: 0.15205664194200944, Validation Accuracy: 0.8474842767295597
Iteration 100, Training Cost: 0.41916887638509953, Validation Cost: 0.4287355451035297, Training Accuracy: 0.8489548213081591, Validation Accuracy: 0.8333333333333334
Iteration 200, Training Cost: 0.4145053869272736, Validation Cost: 0.42686559474252556, Training Accuracy: 0.8459204315576534, Validation Accuracy: 0.8301886792452831
Iteration 300, Training Cost: 0.4120073727265919, Validation Cost: 0.426357568841761, Training Accuracy: 0.8465947403910992, Validation Accuracy: 0.8270440251572327
Iteration 400, Training Cost: 0.41028549476022474, Validation Cost: 0.4262150238253783, Training Accuracy: 0.8459204315576534, Validation Accuracy: 0.8317610062893082
Early stopping at iteration 428 for L1 regularization

Iteration 0, Training Cost: 0.6404308521538322, Validation Cost: 0.6400870868228324, Training Accuracy: 0.15205664194200944, Validation Accuracy: 0.8474842767295597
Iteration 100, Training Cost: 0.4191647018610815, Validation Cost: 0.4287229247540232, Training Accuracy: 0.8489548213081591, Validation Accuracy: 0.8333333333333334
Iteration 200, Training Cost: 0.41450063266732534, Validation Cost: 0.4268515739125931, Training Accuracy: 0.8459204315576534, Validation Accuracy: 0.8301886792452831
Iteration 300, Training Cost: 0.4120023935204613, Validation Cost: 0.4263431649531848, Training Accuracy: 0.8465947403910992, Validation Accuracy: 0.8270440251572327
Iteration 400, Training Cost: 0.4102804627324916, Validation Cost: 0.4262006691516573, Training Accuracy: 0.8459204315576534, Validation Accuracy: 0.8317610062893082
Early stopping at iteration 428 for L2 regularization

Iteration 0, Training Cost: 0.6404308426419837, Validation Cost: 0.6400870424641175, Training Accuracy: 0.15205664194200944, Validation Accuracy: 0.8474842767295597
Iteration 100, Training Cost: 0.4191629508440962, Validation Cost: 0.4287156742743911, Training Accuracy: 0.8489548213081591, Validation Accuracy: 0.8333333333333334
Iteration 200, Training Cost: 0.41449839967485086, Validation Cost: 0.42684249638827404, Training Accuracy: 0.8455832771409305, Validation Accuracy: 0.8301886792452831
Iteration 300, Training Cost: 0.41199973150079666, Validation Cost: 0.426332677075078, Training Accuracy: 0.8465947403910992, Validation Accuracy: 0.8270440251572327
Iteration 400, Training Cost: 0.4102773838773809, Validation Cost: 0.4261888799889034, Training Accuracy: 0.8459204315576534, Validation Accuracy: 0.8317610062893082
Early stopping at iteration 429 for No regularization
```

Experimenting with a learning rate of 1, early stopping occurred around 50 iterations, whereas with a learning rate of 0.01, it happened at approximately 4200 iterations.

The graphs for L1, L2, and no regularization coincide suggesting that regularization has minimal impact on the model's performance in this case. The graphs of the three regularizations combined can be seen below:

